

Action Stories — Template Engine

ENGINEERING OVERVIEW

What we built, how it works, and where it honestly stands — Sep 2026

1 What this project is

"Action Stories" is **one generic rendering engine** that turns raw seller-operations data into ready-to-use decision screens — **Reason → Analyze → Decide → Execute** — for every operating workflow the business runs (assortment, pricing, ads, supply chain, finance, and more). No engineer hand-builds a screen per workflow. The engine reads the shape of the data, decides which component fits, and lays the page out on its own.

26

business workflows covered

105

decision screens generated

5

canonical templates (the whole engine)

19

reusable UI components

50

governed data slots (down from 1,137 raw fields)

99.9%

automated tests passing (1,626 / 1,627)

In one sentence: adding a brand-new 27th workflow tomorrow needs **zero new components** — just two data files. That is the entire point of the architecture, and it is what makes the 26→N growth path cheap.

2 Why this matters for the roadmap

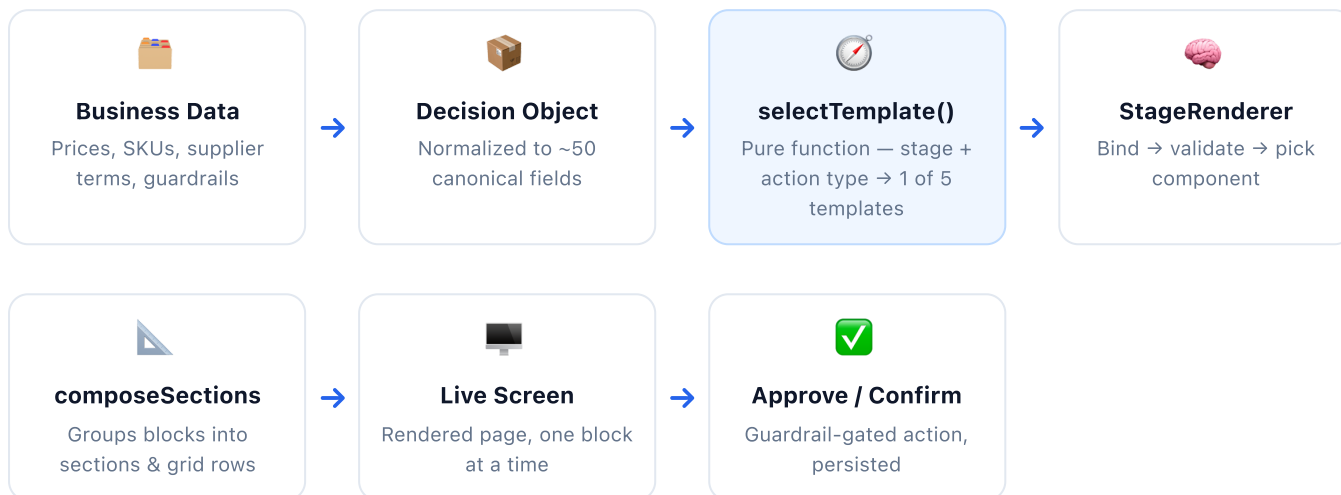
- **Cost:** one engine instead of 105 hand-built screens — new workflows are a data change, not a frontend project.
- **Speed:** a new workflow ships as 2 JSON files; template selection itself is an instant, deterministic 4-row lookup — no scoring engine, no round-trip.
- **Accuracy:** every block is crash-isolated and shape-validated before it renders; the data-fidelity audit (page 4) tracks exactly which fields are real vs. still placeholders.
- **Area covered:** all 26 workflows span 6 operating areas of the business — see page 3 — with the same engine underneath every one of them.

Status, honestly: this is a completed, tested *frontend template engine* running on realistic sample data. It is not yet wired to a live backend, has no auth layer yet, and only the Approve/Confirm action is end-to-end. Full picture on page 5.

From raw data to a live decision screen

One pipeline renders all 105 screens. Nothing here is workflow-specific — the same seven steps run for every one of the 26 workflows.

1 The render pipeline (engine flow)



Key design choice: **the backend never sends a template ID**. It sends business facts (action type, stage, item count); the frontend decides layout. A screen redesign is a frontend deploy, not a cross-team backend change.

2 The business flow every workflow follows



One workflow (Peak Readiness, S10.6) adds a 5th **Live** stage for real-time execution monitoring — reuses the Execute template, no new engine work.

3 What "safe by construction" means

A block with bad/missing data never crashes the screen — it renders a small placeholder and logs a warning instead.

Every block runs inside its own error boundary — one broken card never takes down the rest of the page.

COVERAGE

26 workflows across 6 operating areas

Every workflow below runs on the exact same engine from page 2 — only its data and chosen template differ.



Area	Workflows	Count
Merchandising	Assortment, Launch, Exit, Channel Launch, Item Research	5
Pricing & Margin	Competitive Repricing, Markdown, Margin-protective Repricing	3
Marketing & Discovery	Advertising, Agentic Commerce, Search Visibility, Promo Calendar	4
Supply Chain & Fulfilment	Replenishment, Allocation (x2), Lifecycle, Supplier Performance, Fulfilment Cost	6
Finance & Working Capital	Variance Bridge, Working Capital, Fee Integrity	3
Catalog, Compliance & Ops	Listings, Returns, Account Health, Catalog Integrity, Peak Readiness	5

All 26 workflows are complete end-to-end: every one has all 4 core stages (Reason/Analyze/Decide/Execute) with no gaps — verified by an automated corpus audit, not a manual checklist.

1 Every screen is one of only 5 shapes

Because layout is chosen by **stage**, not by workflow, all 105 screens collapse into 5 reusable templates:

Template	Used for	Screens
reason.v1	Every workflow's Reason stage	26
analyze.compare.v1	Every workflow's Analyze stage	26
decide.slate.v1	Every workflow's Decide stage	26
execute.bridge.v1	Every workflow's Execute stage (and the one Live stage)	27
locked.v1	Access/entitlement short-circuit	as needed

The engineering numbers behind the claims

All figures below are measured directly from the codebase and its automated audits — none are estimates. (This project has no live AI/backend cost or latency data yet — see page 5.)

1 Cost — what consolidation bought us

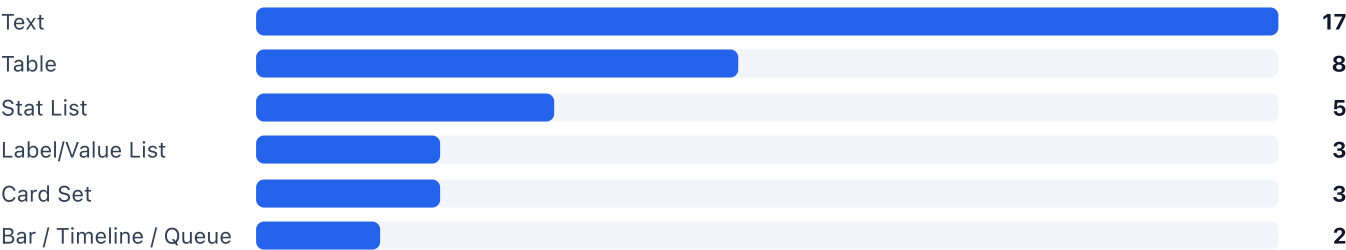


That is a **~94% reduction** in the naming surface an engineer has to understand to change a screen — and it is locked in place by a test, so it cannot silently regress back to one-off naming.

2 Speed — cost of adding the next workflow

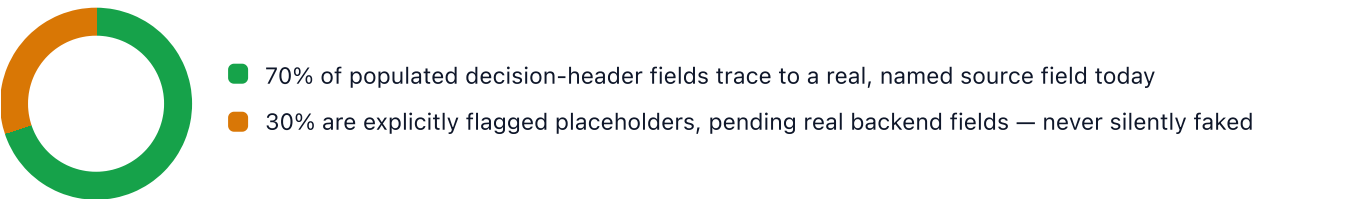
- **New workflow = 2 files.** A data manifest and a fixture — no new component, no new template, no engine change.
- **Template selection is O(1).** A 4-row lookup table keyed on stage — deterministic, synchronous, no network call, no scoring model.
- **New block type (rare) = 1 component + 1 registry line + 1 validator** — the renderer itself is never touched.

3 Component reuse — how the 19 blocks are actually used



Counted as canonical-slot assignments (of 50 total) — e.g. 17 of the 50 governed slots render as plain Text. 6 chart/specialist blocks (line, scatter, waterfall, heatmap, gauge, checklist) cover the remaining analytical edge cases at 1 slot each.

4 Accuracy — tests & data fidelity



1,626 / 1,627 automated tests passing (99.9%) across rendering, data pipeline, and one true end-to-end route test that walks the full fetch → template → render → approve → status-transition path.

What's real today, and what isn't yet

Said plainly, for planning purposes — no part of this is overstated.

1 Solid and shipped

- ✓ Rendering engine: 5 canonical templates, 19 components, 50 governed data slots — covers all 26 workflows / 105 screens.
- ✓ Approve/Confirm action: real end-to-end mutation with loading/error states, persisted so a refresh doesn't lose it.
- ✓ Guardrail gating: decisions can be blocked by a real governance verdict, not a cosmetic flag.
- ✓ Automated safety net: 1,626/1,627 tests, including one full end-to-end route test.

2 Not connected yet — the honest gap list

- ! **No live backend.**
Runs on realistic sample data via one local service file; the swap-in point is built and documented (INTEGRATION.md) but not switched on.
- ! **No auth/access control.**
Anyone with a link can open any screen today.
- ! **Only one action wired.**
Approve/Confirm works end-to-end; other action buttons are illustrative only.
- ✗ **Search bar is non-functional.**
Present in the UI, not implemented.
- ! **No live AI/inference cost or latency data.**
This repo is the presentation layer; per-decision cost/speed/accuracy for an actual model pipeline will come from wherever that pipeline lands.

3 Recommended next steps

#	Step	Why it's next
1	Connect a real data source through the existing service swap-in point	Unblocks everything else; architecture already assumes it
2	Add authentication / access control	Currently zero access restriction — a hard blocker before any real rollout
3	Wire the remaining action buttons to real operations	Only Approve/Confirm does real work today
4	Re-run the data-fidelity audit against live fields	Moves the 30% placeholder figure toward 0% with real evidence, not estimates

Bottom line: the hard, reusable part — one engine covering 26 business areas instead of 105 one-off screens — is done and tested. What's left is systems-integration work (backend, auth, remaining actions), not more frontend design.